	School of ICT **Sirindhorn International Institute of Technology**
CSS225 Operating System	**Homework 6: Resource Allocation and Deadlocks**

Part 1: Multiple-Choice Questions

1. _____ is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.
   A) Preemption
   B) A deadlock
   C) Page replacement
   D) Swapping

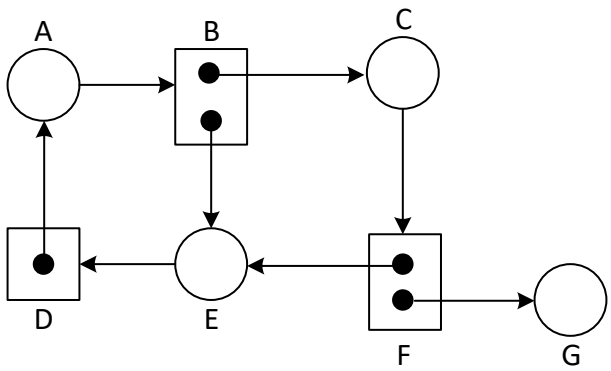
2. Which one is INCORRECT about resource types and instances?
   A) Consider a system with four different models of printers. Here, there are four resource types.
   B) The resources may be partitioned into several types (or classes), each consisting of some number of identical instances.
   C) Consider a system with three identical CPUs. Here, there are three instances of the resource type named CPU.
   D) All answers are correct.

3. Which one is INCORRECT about the necessary conditions in a deadlock?
   A) There are four necessary conditions.
   B) One necessary condition for deadlock is *no preemption*, which states that at least one resource must be held in a non-sharable mode.
   C) One necessary condition for deadlock is *hold and wait*, which states that a process must be holding one resource and waiting to acquire additional resources.
   D) All answers are correct.

4. Which one is INCORRECT about a resource-allocation graph?
   A) A resource-allocation graph consists of a set of vertices and a set of edges.
   B) The set of vertices is partitioned into two different types of nodes: processes and resource types.
   C) Each resource-type node has only one instance.
   D) A request edge is denoted by an arrow pointing from a process to a resource type.

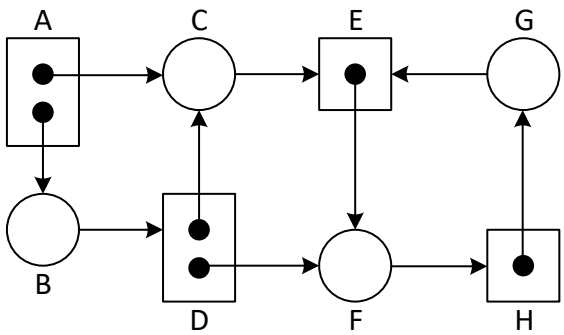
5. Which one is CORRECT about an assignment edge?
   A) An assignment edge is a directed edge from a process to a resource type.
   B) An assignment edge is a directed edge from an instance in a resource type to a process.
   C) An assignment edge is a directed edge from a process to an instance in a resource type.
   D) An assignment edge is a directed edge from an instance in a resource type to a resource type.

6. Consider the following resource-allocation graph. Which one is INCORRECT?



A) The process C is holding an instance of the resource type B and is waiting for an instance of the resource type F.
B) A set of processes is {A, C, E, G}.
C) The system is in a deadlock.
D) All answers are correct.

7. Consider the following resource-allocation graph. Which one is INCORRECT?



A) The process B is holding an instance of the resource type A and is waiting for an instance of the resource type D.
B) A set of processes is {B, C, E, F, G}.
C) The system is in a deadlock.
D) All answers are correct.

8. Most of operating systems _____.
A) ignore the deadlock problem and pretend that deadlocks never occur in the system
B) allow the system to enter a deadlocked state, detect it, and recover
C) use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlocked state
D) No correct answer

9. Which one of the following actions could prevent a deadlock?
A) Allow the mutual exclusion condition.
B) Allow the hold-and-wait condition.
C) Allow the preemption condition.
D) Allow the circular-wait condition.

10. There will be no _____ if all resources are sharable.
    A) mutual exclusion
    B) hold-and-wait
    C) preemption
    D) circular-wait

11. When should we apply the safety algorithm?
    A) To find out whether resource assignments should be preempted.
    B) To find out whether a system is in a safe state.
    C) To find out whether a process should be terminated.
    D) To find out whether a system is in the circular-wait condition.

12. If a system does NOT employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. In this environment, what may the system provide/do?
    A) An algorithm to check whether the system is in a safe state.
    B) An algorithm to recover from the deadlock.
    C) An algorithm to grant resource requests.
    D) No correct answer.

13. If deadlock is controlled by the banker's algorithms (safety algorithm and resource-request algorithm), which of the following actions (on the variables in these algorithms) can be done safely (without introducing the possibility of deadlock)?
    A) Increase the variable Max and the number of processes
    B) Increase the variable Max and decrease the variable Available
    C) Decrease the variable Max and increase the number of processes
    D) Decrease the variable Max and increase the variable Available

14. Which one is INCORRECT about the process termination?
    A) It is a recovery method from a deadlock.
    B) We will preempt some resources from processes and give these resources to other processes.
    C) We will eliminate one process at a time and check whether a deadlock has been eliminated.
    D) All answers are correct.

15. What will/could NOT happen when we invoke/call a deadlock-detection algorithm frequently?
    A) Overhead in computation time decreases.
    B) We can detect a deadlock sooner.
    C) The number of processes involved in a deadlock is small.
    D) No correct answer.

16. Consider the following resource-allocation state. Determine whether the system is in a deadlock state and the processes involved in this deadlock?

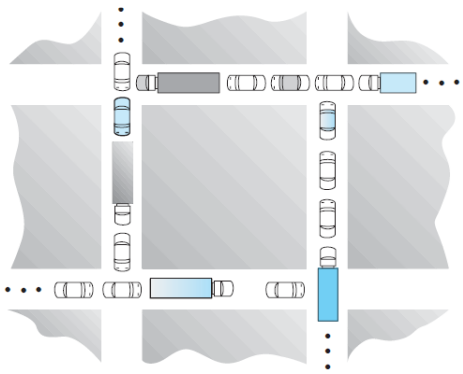
	Allocation			**Request**		
	A	**B**	**C**	**A**	**B**	**C**
Process P0	1	0	1	2	0	1
Process P1	0	1	2	1	2	1
Process P2	2	3	1	2	1	0

Available		
A	**B**	**C**
1	0	1

    A) The system is not in a deadlock.
    B) The system is in a deadlock, which consists of P0, P1, and P2.
    C) The system is in a deadlock, which consists of P0 and P1.
    D) The system is in a deadlock, which consists of P1 and P2.

Part 2: Short-Answer Questions

1. (Exercise7.1) Consider the traffic deadlock depicted in the figure shown below.



a) Show that the four necessary conditions for deadlock hold in this example.

The four necessary conditions for a deadlock are (1) mutual exclusion; (2) hold-and-wait; (3) no preemption; and (4) circular wait. The mutual exclusion condition holds since only one car can occupy a space in the roadway. Hold-and-wait occurs where a car holds onto its place in the roadway while it waits to advance in the roadway. A car cannot be removed (i.e. preempted) from its position in the roadway. Lastly, there is indeed a circular wait as each car is waiting for a subsequent car to advance. The circular wait condition is also easily observed from the graphic.

b) State a simple rule for avoiding deadlocks in this system.

A simple rule that would avoid this traffic deadlock is that a car may not advance into an intersection if it is clear it will not be able immediately to clear the intersection.

2. (Exercise 7.6) In a real computer system, neither the resources available nor the demands of processes for resources are consistent over long periods (months). Resources break or are replaced, new processes come and go, and new resources are bought and added to the system. If deadlock is controlled by the banker's algorithm, which of the following changes can be made safely (without introducing the possibility of deadlock), and under what circumstances?

   a) Increase Available (new resources added).

This could safely be changed without any problems.

   b) Decrease Available (resource permanently removed from system).

This could have an effect on the system and introduce the possibility of deadlock as the safety of the system assumed there were a certain number of available resources.

   c) Increase Max for one process (the process needs or wants more resources than allowed).

This could have an effect on the system and introduce the possibility of deadlock.

   d) Decrease Max for one process (the process decides it does not need that many resources).

This could safely be changed without any problems.

   e) Increase the number of processes.

This could be allowed assuming that resources were allocated to the new process(es) such that the system does not enter an unsafe state.

   f) Decrease the number of processes.

This could safely be changed without any problems.

3. (Exercise 7.13) Consider the following snapshot of a system:

	Allocation	*Max*	*Available*
	A B C D	*A B C D*	*A B C D*
P_0	2 0 0 1	4 2 1 2	3 3 2 1
P_1	3 1 2 1	5 2 5 2	
P_2	2 1 0 3	2 3 1 6	
P_3	1 3 1 2	1 4 2 4	
P_4	1 4 3 2	3 6 6 5	

Answer the following questions using the banker's algorithm:

a) Illustrate that the system is in a safe state by demonstrating an order in which the processes may complete.

Step 0: Note that Max, Allocation, and Need are matrices.

Available = [3 3 2 1]
Max = [4 2 1 2; 5 2 5 2; 2 3 1 6; 1 4 2 4; 3 6 6 5]
Allocation = [2 0 0 1; 3 1 2 1; 2 1 0 3; 1 3 1 2; 1 4 3 2]
Need = [2 2 1 1; 2 1 3 1; 0 2 1 3; 0 1 1 2; 2 2 3 3]

Step 1: Declare 2 variables.

Work = [3 3 2 1]
Finish = [False False False False False]

Step 2 and Step3: Check process i = 0, 1, 2, …, 4 (and repeat if necessary) to find out whether the system is in a safe state.

i = 0: Need0 <= Work
Finish[0] = True
Work = [3 3 2 1] + Allocation0 = [3 3 2 1] + [2 0 0 1] = [5 3 2 2]
Finish = [True False False False False]

i = 1: Need1 > Work

i = 2: Need2 > Work

i = 3: Need3 <= Work
Finish[3] = True
Work = [5 3 2 2] + Allocation3 = [5 3 2 2] + [1 3 1 2] = [6 6 3 4]
Finish = [True False False True False]

i = 4: Need4 <= Work
Finish[4] = True
Work = [6 6 3 4] + Allocation4 = [6 6 3 4] + [1 4 3 2] = [7 10 6 6]
Finish = [True False False True True]

i = 1: Need1 <= Work
Finish[1] = True
Work = [7 10 6 6] + Allocation1 = [7 10 6 6] + [3 1 2 1] = [10 11 8 7]
Finish = [True True False True True]

i = 2: Need2 <= Work
Finish[2] = True
Work = [10 11 8 7] + Allocation1 = [10 11 8 7] + [2 1 0 3] = [12 12 8 10]
Finish = [True True True True True]

Step 4: All processes can finish their jobs. The system is in a safe state. According to the above, we have the following safe sequence: < P0, P3, P4, P1, P2 >.

b) If the process P1 requests for (1, 1, 0, 0), can the request be granted immediately? Note that (1, 1, 0, 0) means 1 instance of A, 1 instance of B, 0 instance of C, and 0 instance of D.

- Assume that the OS grants this request to P1 and determines whether the system will be in a safe state or not.

  Request1 [1 1 0 0]
- Update 4 variables: Note that those related to the request of P1 were updated (highlighted).

  Available = [2 2 2 1]
  Max = [4 2 1 2; 5 2 5 2; 2 3 1 6; 1 4 2 4; 3 6 6 5]
  Allocation = [2 0 0 1; 4 2 2 1; 2 1 0 3; 1 3 1 2; 1 4 3 2]
  Need = [2 2 1 1; 1 0 3 1; 0 2 1 3; 0 1 1 2; 2 2 3 3]

Now, we run the safety algorithm.
Step 1: Declare 2 variables.
Work = [2 2 2 1]
Finish = [False False False False False]

Step 2 and Step3: Check process i = 0, 1, 2, …, 4 (and repeat if necessary) to find out whether the system is in a safe state.

i = 0: Need0 <= Work
Finish[0] = True
Work = [2 2 2 1] + Allocation0 = [2 2 2 1] + [2 0 0 1] = [4 2 2 2]
Finish = [True False False False False]

i = 1: Need1 > Work

i = 2: Need2 > Work

i = 3: Need3 <= Work
Finish[3] = True
Work = [4 2 2 2] + Allocation3 = [4 2 2 2] + [1 3 1 2] = [5 5 3 4]
Finish = [True False False True False]

i = 4: Need4 <= Work
Finish[4] = True
Work = [5 5 3 4] + Allocation4 = [5 5 3 4] + [1 4 3 2] = [6 9 6 6]
Finish = [True False False True True]

i = 1: Need1 <= Work
Finish[1] = True
Work = [6 9 6 6] + Allocation1 = [6 9 6 6] + [4 2 2 1] = [10 11 8 7]
Finish = [True True False True True]

i = 2: Need2 <= Work
Finish[2] = True
Work = [10 11 8 7] + Allocation1 = [10 11 8 7] + [2 1 0 3] = [12 12 8 10]
Finish = [True True True True True]

Step 4: All processes can finish their jobs. The system is in a safe state. According to the above, we have the following safe sequence: < P0, P3, P4, P1, P2 >.

<u>As a result, the OS can allow this request of P1.</u>

c) If the process P4 requests for (0, 0, 2, 0), can the request be granted immediately? Note that (0, 0, 2, 0) means 0 instance of A, 0 instance of B, 2 instance of C, and 0 instance of D.

- Assume that the OS grants this request to P4 and determines whether the system will be in a safe state or not.

  Request4 [0 0 2 0]
- Update 4 variables: Note that those related to the request of P4 were updated (highlighted).

  Available = [3 3 0 1]
  Max = [4 2 1 2; 5 2 5 2; 2 3 1 6; 1 4 2 4; 3 6 6 5]
  Allocation = [2 0 0 1; 3 1 2 1; 2 1 0 3; 1 3 1 2; 1 4 5 2]
  Need = [2 2 1 1; 2 1 3 1; 0 2 1 3; 0 1 1 2; 2 2 1 3]

Now, we run the safety algorithm.
Step 1: Declare 2 variables.
Work = [3 3 0 1]
Finish = [False False False False False]

Step 2 and Step3: Check process i = 0, 1, 2, …, 4 (and repeat if necessary) to find out whether the system is in a safe state.
i = 0: Need0 > Work
i = 1: Need1 > Work
i = 2: Need2 > Work
i = 3: Need3 > Work
i = 4: Need4 > Work
Since, in the first round here, no process terminated, the number of resources in Available is still the same. If we repeat i = 0, 1, …, 4 again, we still have the same result.
Step 4: We can conclude that the system is in the deadlock.

<u>As a result, the OS should NOT allow this request of P4</u>.